ReSpec — Execution Plan and Task Sheets

Team: **Divyansh** (lead), **Aarvi**, **Bhavya**, **Riya**. Baseline: the attached 'ReSpec — Critical Review and Build Specification' (called the **Spec** below). Status date: Sat Oct 10, 2026 (DTU build day). Tags as in the Spec: [V] verified · [A] assumption · [E] estimate · [S] synthetic · [U] unverified.

Deadline status (checked just now): the official schedule page still says the submission hours are being finalised; Sunday Oct 11 is the last day to submit. Your internal '8 PM IST on Oct 11' is **not** official. Plan: submit by **15:00 on Oct 11**; recheck the schedule page and Discord at 08:00 and 12:00 on Oct 11.

Unknown: which Spec tasks from 'Oct 9 tonight' (repo, contracts, fixtures, cached Textract JSON) are already done. Block 0 below starts with a 10-minute audit. If the clock is already past a block's start, compress it by cutting what its fallback line says.

---

1. ReSpec in simple words

The problem. Delhi makes about 6,054 tonnes of construction and demolition (C&D) waste a day and has plants that turn it into aggregate, pavers and kerbs. Yet MCD reported only 14% of recycled products were used in 2024-25. Since April 1, 2026, large projects (20,000 m² or more) and road projects must use a minimum share of recycled material: 5% of total construction material this year. [V per Spec]

The user. A quantity surveyor or procurement engineer. Their main document is a **BOQ — Bill of Quantities** — a long table listing every work item with a unit and quantity (for example, 'plain concrete under foundations, 120 m³'). Today they must read hundreds of lines by hand to see where recycled material is allowed.

What ReSpec does (illustrative numbers invented for explanation only). A campus builds a block and a road. The BOQ has: lean concrete 100 m³, M20 reinforced concrete 80 m³, M30 columns 50 m³, paver blocks 800 m², granular sub-base 300 m³.

1. **Read it.** ReSpec extracts the table. **Amazon Textract** is an AWS service that reads a scanned or PDF document and returns its tables as structured data.
2. **Check it.** A person fixes any misread cell.
3. **Judge each line with fixed rules. Deterministic rules** means the same line always gives the same answer, because the answer comes from written rules with a source (for example IS 383:2016: recycled concrete aggregate up to 25% in plain concrete, 20% in

reinforced concrete only up to M25). Lean concrete: eligible. M20 reinforced: eligible with conditions. M30 columns: **MANUAL_REVIEW** (the standard stops at M25). Lines without a sourced rule also go to MANUAL_REVIEW. No AI decides eligibility.

4. **Compliance gap.** ReSpec adds up how much material could safely be recycled and compares it to the 5% target: 'This BOQ can reach X% against a 5% target.' This matters because the standards cap recycled aggregate in structural concrete, so most of the target must come from non-structural items like pavers and sub-base. Showing the gap honestly is the product's insight.
5. **Human approves** each recommendation; ReSpec exports a draft procurement plan listing product, quantity and nearby plants (straight-line distance; stock must be confirmed by phone).
6. **Delivery verification.** When material arrives, the user records the delivered tonnes and uploads a weighbridge slip or invoice. The ledger moves ELIGIBLE → PLANNED → ORDERED → **VERIFIED**. **Verified tonnes** are tonnes with a delivery record and an attached evidence file; only those are reported as achieved. Everything earlier is 'modelled'.

Why AWS. S3 stores the BOQ and evidence; Textract reads the document; **Step Functions** (an AWS workflow coordinator) runs the multi-step extraction visibly; Lambda runs the logic; DynamoDB stores each job's state changes; CloudWatch records logs; Amplify hosts the app.

---

2. End-to-end workflow and minimum product scope

```
Upload BOQ (PDF) or load CSV/XLSX/paste rows
  → Step Functions: [Textract if PDF] → Normalize → Classify → Persist
(NEEDS_REVIEW)
  → User corrects and confirms table (READY)
  → Evaluate: rules + gap vs target
  → Approve / reject (only ELIGIBLE and CONDITIONAL lines)
  → Generate plan (PLANNED) → enter PO ref (ORDERED)
  → Record delivery + evidence (VERIFIED or FLAGGED) → Impact totals
```

Must ship (unchanged from the Spec): 6 screens (S1–S6); PDF via Textract plus CSV/XLSX/paste fallback; review table; recommendations with source clause and status; gap vs target with visible assumptions; approval; draft plan export (CSV + printable PDF, marked DRAFT); four-state ledger gated by evidence; error, empty and loading states; deployed on AWS in ap-south-1.

Cut (never build): CO2/energy numbers, marketplace, payments, live stock, CPCB submission, optimiser beyond nearest-facility, image classification, chatbot, mobile app, Cognito, multi-tenant admin. **Bedrock stays off** (feature flag; keyword classifier is the default).

Safety rules that cannot be negotiated: unsourced or unknown-grade or structural-beyond-limit lines → MANUAL_REVIEW or NOT_ALLOWED; those lines cannot be approved; every recommendation shows its source clause; only evidenced deliveries raise verified tonnes; nothing is evaluated until the user confirms the table.

---

3. Responsibility matrix

	Primary ownership	Secondary	Why this split
Divyansh	Architecture, contracts, safety-critical core: gap calculator, ledger state machine, impact totals; IAM and state-machine review; integration; mentors; demo and claims	Co-owns Step Functions definition with Bhavya; reviews every rule row	Keeps the highest-risk code and decisions with the lead; code is small and pure, so it does not crowd out supervision
Bhavya	AWS: SAM template, S3, DynamoDB, HTTP API, Lambda handlers, Step Functions + Textract, deployment, CloudWatch, budget alarm, AWS proof pack	Plan CSV export; evidence presign	Independent of rules and UI once contracts are frozen
Riya	Domain engine: rules.json, conversions.json, facilities.json, normaliser, classifier, rules evaluator, unit tests, labelled set and evaluation script	Source register entries	Pure TypeScript functions, testable without AWS

	Primary ownership	Secondary	Why this split
Aarvi	Web app S1–S6 against fixtures, browser CSV/XLSX parse, printable plan, error states; README, SOURCES.md, THIRD_PARTY_NOTICES, AI-tool disclosure; demo assets and video editing	UI polish; checklists; Builder Center blog draft	Real, demonstrable ownership with explicit acceptance criteria and no infrastructure or safety decisions

Mapping to the Spec's E1–E4: E1 → Aarvi, E2 → Bhavya, E3 → Riya (rules/evaluate) + Divyansh (gap/ledger), E4 → Riya (normaliser/classifier/eval) + Aarvi (docs/video).

---

4. Individual task sheets

4.1 Divyansh — Team lead

#	Task	Deliverable	Acceptance
D1	State audit + contract freeze (Block 0)	/contracts/v1 types, /fixtures for every endpoint, /docs/DECISIONS.md with Section 5 decisions D1–D12 resolved	Fixtures validate against schemas in a passing test; all four confirm in chat
D2	`/core/gap`: computeGap(lines, recs, project)	Gap object per contract incl. applicability, denominator, numerator, assumptions, topLines	≥8 tests: building ≥20,000 m² → 5% target; road → Third Schedule; below threshold → not applicable; metal/wood excluded; missing mass excluded and listed

#	Task	Deliverable	Acceptance
D3	`/core/ledger`: pure reducer over append-only events	Totals eligible/planned/ordered/verified in tonnes; transition guards	⩾10 tests incl. illegal transitions (approve a MANUAL_REVIEW rec, order before plan, verify without evidenceKey, delivery > planned + 5%, duplicate slip number)
D4	Review Step Functions definition and IAM with Bhavya	Approved ASL and role policies	No wildcard resources; each Lambda role scoped to its prefix; Textract Start/Get only on pipeline role
D5	Review every rules.json row and conversions value	Each rule has source URL + clause or is MANUAL_REVIEW	No ELIGIBLE rule without a source
D6	Integration owner at C1–C4	Main branch green; checkpoint demos	Checkpoint criteria in Section 7 met
D7	Mentor sessions (Section 9)	Notes in /docs/MENTOR_NOTES.md	Open questions closed or downgraded to MANUAL_REVIEW
D8	Demo and claims audit	Script, final README claims check, narration recording	Every number on screen traceable to a run; synthetic items labelled

Do not start: D2 before Riya's evaluate outputs per-line mass (agree the interface in D1); D3 endpoints wiring before Bhavya's table exists (reducer is pure, start immediately). **Fallback:** if D2 slips, ship gap with total only (no topLines) and say so; never fake the number. **Handoff:** merge to main; post the function signatures in chat; Bhavya imports from /core.

4.2 Bhavya — AWS and backend

#	Task	Deliverable	Acceptance
B1	SAM template: S3 (block public access, CORS to app origin), DynamoDB (PK/SK on-demand), HTTP API, Lambdas (api, startPipeline, normalize, classify, persist), log retention 14 days, AWS Budgets alarm at $10	/infra/template.yaml	`sam deploy` from a clean checkout creates the stack in ap-south-1
B2	All Section 7 endpoints returning fixture-shaped responses	Deployed API URL	A contract test hits every endpoint and validates the schema
B3	Step Functions: Choice(source) → Textract StartDocumentAnalysis(TABLES) → Wait 5s → GetDocumentAnalysis loop with pagination → Normalize → Classify → Persist; Catch → status ERROR	State machine + Textract JSON cached at textract/<filehash>.json	Demo PDF reaches NEEDS_REVIEW; rows path skips Textract and still runs Normalize→Persist through the same machine
B4	DynamoDB persistence: jobs, lines, recs, decisions, deliveries, plans	Handlers write/read per the data model	Re-running a step does not duplicate items (idempotent)
B5	evaluate endpoint calling Riya's evaluate and Divyansh's computeGap	Recs and gap stored	Fixture BOQ returns the expected statuses
B6	decisions, orders, evidence-url, deliveries, impact using the ledger reducer; S3 HEAD check on evidenceKey	Endpoints live	Delivery without an existing S3 object is FLAGGED and does not raise verified tonnes

#	Task	Deliverable	Acceptance
B7	plan endpoint writes CSV to S3 and returns a presigned URL (expires 5 min)	exportUrls.csv	Opens in Excel; contains source clause column and DRAFT marker
B8	Amplify hosting from the repo, env vars, .env.example, deploy script	Public URL	Loads signed-out in a clean browser
B9	AWS proof pack	/docs/aws-proof/ with screenshots and execution ARNs	Contains one succeeded and one failed execution, S3 object, DynamoDB items, CloudWatch lines by jobId

Do not start: B5 until Riya's evaluate signature is merged; B6 until Divyansh's ledger reducer is merged (stub with fixture in the meantime). **Fallback:** if Textract merged-cell output is unusable on the demo PDF, keep the machine and run the rows path for the live demo; show a real Textract run on a simpler page for the AWS proof, and say so in the video. If ASL is blocked >40 min, escalate to the AWS mentor. **Handoff:** post the API URL, a curl example per endpoint and the stack outputs in chat.

4.3 Riya — Domain engine and evaluation

#	Task	Deliverable	Acceptance
R1	/data/rules.json from Spec Section 5 sourced rules only (R-LEAN, R-PLAIN, R-RCC, prestressed exclusion, Note 2 condition, schedules) plus a default MANUAL_REVIEW rule per category	Rule records with id, category, condition, allowed_product, max_share, basis, status, source_url, clause, last_checked, reviewed_by	Every ELIGIBLE or CONDITIONAL rule has a URL and clause; a schema test enforces it

#	Task	Deliverable	Acceptance
R2	/data/conversions.json and /data/facilities.json	Unit table, density/thickness factors flagged [E] with override field; facilities from the DPCC table, Rani Khera = Mundka (one site), each marked [U]	No mass computed without a factor; no stock fields exist
R3	/core/normalize	Unit normalisation to m3, m2, m, nos, kg, t; grade parsing (M10–M30, nominal mixes like 1:4:8); structural flag (RCC, reinforced, prestressed, retaining, water-retaining); outlier flag (>10× median per unit)	Unknown unit → flag + MANUAL_REVIEW
R4	/core/classify (keyword/fuzzy)	10 categories with confidence	Unknown → 'ineligible/manual review'
R5	/core/evaluate	Rec per line; per-line massT and excluded flag for the denominator; facility options by haversine	Same input always gives identical output (snapshot test)
R6	Unit tests	≥25 including negative controls	`npm test` green in CI
R7	/eval labelled set (50 lines, 10 negative controls) and script	Metrics file: extraction accuracy, classification precision/recall, negative-control result	Script reproduces numbers; zero ELIGIBLE/CONDITIONAL on negative controls

Do not start: R5 before R1–R4 interfaces are merged; R7 before the demo BOQ is chosen. **Fallback:** if grade parsing is unreliable, return unknown grade → MANUAL_REVIEW (safe default). If density factors are missing, show volume only. **Handoff:** PR with signatures, a

README in /core, and a list of rules needing the civil mentor in /docs/OPEN_QUESTIONS.md.

4.4 Aarvi — Web app, documentation and demo assets

#	Task	Deliverable	Acceptance
A1	Scaffold /web (Vite + React + TypeScript), router for S1–S6, typed API client with mock mode (`VITE_MOCK=1` reads /fixtures)	Running app	`npm run dev` shows all six routes from fixtures
A2	S1 New project form	Name, type, built-up area, site location, FY	Message shows 'mandatory' when area ⩾20,000 m² or type is road, otherwise 'not mandatory under the 2025 Rules'
A3	S2 Upload	File picker, paste rows, CSV/XLSX parse in browser (SheetJS), 'Load sample BOQ' button, progress from GET /jobs/{id}	Status updates within 5 s of a stage change
A4	S3 Review table	Confidence colours, inline edit via PATCH, confirm button	Edits persist after reload; low-confidence cells highlighted
A5	S4 Recommendations	Status badge, product, max share, source clause, assumptions; approve/reject	Approve disabled for MANUAL_REVIEW and NOT_ALLOWED; no row without a source
A6	S5 Gap vs target	Target %, modelled %, denominator, assumptions, driving lines	Assumptions visible without scrolling

#	Task	Deliverable	Acceptance
A7	S6 Plan and ledger	Plan rows, PO ref input, evidence upload via presigned URL, delivery form, four-state totals, flags	Verified total changes only after a delivery with evidence
A8	Printable plan (print stylesheet with DRAFT watermark)	PDF via browser print	Contains source clause per row
A9	Loading, empty and error states on every screen	Components	Extraction error offers CSV/paste
A10	README skeleton, SOURCES.md, THIRD_PARTY_NOTICES.md, AI-tool disclosure file, Builder Center blog draft	Files in repo	Each entry has URL, access date, licence, verified/unverified flag
A11	Demo support	Screenshots, timing sheet, video editing	Final cut ⩽2:55

Do not start: wiring to the real API before Bhavya posts the API URL (stay in mock mode); any change to a contract field (ask Divyansh). **Fallback:** if S6 slips, build ledger display first and evidence upload second; keep CSV export from the API as the plan output. **Handoff:** PR with screenshots of each screen in mock mode and in real mode.

---

5. Architecture, contracts and integration rules

Architecture and API contracts are exactly as in Spec Sections 6–8 (TypeScript monorepo: /web, /api, /core, /infra, /data, /eval, plus /contracts, /fixtures, /docs). Region ap-south-1.

Unresolved decisions and inconsistencies in the Spec (proposed defaults; Divyansh approves in Block 0)

#	Issue	Proposed default
D1	Spec's timeline is dated Oct 9; today is Oct 10	Use Section 7; audit what exists first

#	Issue	Proposed default
D2	CSV/XLSX path: does it still use Step Functions?	Yes — same machine, Choice skips Textract, so AWS stays visible on the fallback
D3	Gap needs mass for every line, but Line and Rec only carry modelled mass for recommendations	Add optional `massT`, `massBasis`, `excludedFromDenominator` to Line (additive, non-breaking)
D4	Ledger vs delivery states: deliveries return VERIFIED or FLAGGED, but FLAGGED is not a ledger state	Totals are derived per quantity from events; FLAGGED deliveries never count; a rec shows VERIFIED once any non-flagged evidenced delivery exists, with verified tonnes tracked separately
D5	When is a rec PLANNED?	APPROVE records the decision; POST /plan sets approved recs to PLANNED, so planned tonnes exist only after a plan is generated
D6	Can MANUAL_REVIEW or NOT_ALLOWED lines be approved?	No override in the MVP (safest)
D7	Can CONDITIONAL be approved?	Yes, only with the approver acknowledging each condition
D8	Editing lines after evaluate	Resets status to NEEDS_REVIEW and discards recs/decisions; lines lock once any order or delivery event exists
D9	Auth	Shared demo passcode in an `x-demo-passcode` header validated by Lambda; value in env, never in the repo; say so in the video
D10	Exports	CSV written to S3 by the plan Lambda (presigned URL); PDF via browser print
D11	Concrete aggregate fraction and density seed values are unspecified in the Spec	Riya proposes values in conversions.json flagged [E] with source or reasoning; Divyansh approves; mentor reviews
D12	R-DELHI-2016 and R-CPWD percentages	Shown as context only; they do not change eligibility

Other [A]/[U] items to confirm with mentors: gap denominator base and whether the target counts mass or volume; Gazette text of the schedules (the Spec used secondary summaries); facility list and what each plant sells; current validity of the 2016 advisory; Bedrock availability (not needed).

Contract freeze. Contracts live in /contracts/v1 as TypeScript types plus Zod schemas; /fixtures hold one example response per endpoint. Both fixtures and Lambda responses are validated against the schemas in tests. Freeze at the end of Block 0. After that, any change needs a PR labelled `contract-change` that updates the fixture first, is announced in chat and is approved by Divyansh; additive optional fields only; no changes after 16:00 except bug fixes.

Pull requests and reviews

- Trunk-based: short-lived branches `<owner>/<task>`; PRs under about 300 lines; rebase on main before merge; no force-push to main.
- CODEOWNERS: /core/gap, /core/ledger, /data/rules.json, /data/conversions.json, /infra/** require Divyansh. Everything else needs one approval from someone other than the author.
- Routine UI and documentation PRs: Aarvi may self-merge after CI is green. Divyansh is not required.
- CI: lint, type-check, unit tests, contract tests on every PR.
- Directory ownership prevents conflicts: web → Aarvi, infra/api → Bhavya, core/data/eval → Riya, core/gap + core/ledger + contracts → Divyansh.

Cadence. Ten-minute standup at 10:30, 12:00, 14:00, 16:00, 18:00 and 20:00. Main must be green at each. Integrate to main at least every two hours.

Progress report (post in chat at each standup)

```
DONE: <what, with commit/PR link>
NEXT: <next task id and ETA>
BLOCKED: <what, on whom, since when>
EVIDENCE: <screenshot, URL, test output or execution ARN>
```

Escalation. Blocked 20 minutes → post BLOCKED with evidence. 40 minutes → Divyansh pairs or reassigns. AWS, IAM or Textract issues → ask an AWS mentor immediately. A rule you cannot source → default MANUAL_REVIEW and log it in /docs/OPEN_QUESTIONS.md; do not wait.

Divyansh must approve: contract changes; each rule and conversion row; state transitions; impact and gap formulas; IAM policies; DynamoDB schema; final deployment; every number in the demo and README. **Engineers decide alone:** component structure and styling; helper functions; test names; log format (jobId required); libraries from the allowlist (React, Vite, Zod, Vitest, SheetJS, AWS SDK) with licences logged in THIRD_PARTY_NOTICES.

---

6. Deliverables and submission checklist

Mandatory for submission [V, official rules and judging page as read earlier]: public repo with code written on or after Oct 8; YouTube demo (public or unlisted) of 3 minutes or less showing AWS; short write-up (problem, build, where AWS fits); licences and credits for libraries/templates/APIs; AI coding tools disclosed; every member's student verification. **Everything marked (rec.) below is recommended, not mandatory.** The exact submission form and closing time are not yet published — recheck.

A. Working product

Deliverable	Owner / support	Location	Deadline	Acceptance
Deployed web app	Bhavya / Aarvi	Amplify URL	C3, 16:00	Loads signed-out; full flow runs on the demo BOQ
Upload + CSV/XLSX/paste fallback	Aarvi / Bhavya	/web S2, /api	C2, 12:00	PDF and CSV both reach NEEDS_REVIEW
Extraction + review table	Bhavya, Aarvi	S3, /api	C2, 12:00	Real Textract lines editable
Sourced recommendations	Riya / Divyansh	/core/evaluate, S4	C3, 16:00	Each row shows status and source clause; unsourced → MANUAL_REVIEW
Compliance-gap calculation	Divyansh / Riya	/core/gap, S5	C3, 16:00	Assumptions visible; matches hand calculation on fixture

Deliverable	Owner / support	Location	Deadline	Acceptance
Approval flow	Aarvi / Divyansh	S4	C3	Cannot approve MANUAL_REVIEW or NOT_ALLOWED
Draft plan export	Bhavya (CSV), Aarvi (print)	S3 export, S6	C3	DRAFT marker, source clause column
Delivery evidence + four-state ledger	Divyansh / Bhavya, Aarvi	/core/ledger, S6	C3	Verified rises only with evidence
Error, empty, loading states	Aarvi	/web	18:00	Each screen demonstrates all three

B. AWS proof

Deliverable	Owner / support	Location	Deadline	Acceptance
Architecture diagram	Divyansh / Aarvi	/docs/architecture.png, README	18:00	Matches deployed resources
Real Textract execution	Bhavya	/docs/aws-proof	C2, 12:00	Output cached in S3
Step Functions history (success + failure)	Bhavya	screenshots, ARNs	18:00	Both paths visible
DynamoDB state transitions	Bhavya	screenshots	18:00	Job moves through statuses
CloudWatch logs by jobId	Bhavya	screenshots	18:00	One jobId traced end to end

Deliverable	Owner / support	Location	Deadline	Acceptance
Least-privilege IAM summary	Bhavya / Divyansh	/docs/IAM.md	18:00	No wildcards
Cost controls	Bhavya	/docs/COST.md	18:00	Budget alarm, caching, free-tier note
One-line justification per service	Divyansh	README	20:00	Matches Spec Section 6

C. Hackathon submission

Deliverable	Owner / support	Location	Deadline	Acceptance
Public repo, history from Oct 8	Divyansh	GitHub	now	Anyone signed-out can clone
Demo video ⩽3:00, AWS shown	Divyansh / Aarvi	YouTube	Oct 11 14:00	Plays signed-out; length under 3:00
Write-up	Divyansh / Aarvi	submission form + README	Oct 11 12:00	Problem, build, AWS, honest impact
Licences, credits, third-party notices	Aarvi / Riya	THIRD_PARTY_NOTICES.md	20:00	Every dependency listed
AI-tool disclosure	Aarvi (all provide input)	/docs/AI_TOOLS.md	20:00	Names tools and where used
Student verification, all four	each member	Builder Center	now	Each confirms in chat

Deliverable	Owner / support	Location	Deadline	Acceptance
(rec.) Builder Center blog	Aarvi / Divyansh	Builder Center	Oct 11 12:00	Published

D. Engineering and evaluation

Deliverable	Owner / support	Location	Deadline	Acceptance
README, setup and deploy steps	Aarvi / Bhavya	/README.md	20:00	Fresh clone deploys using README only
.env.example, SAM template	Bhavya	repo root, /infra	12:00	No secrets committed
API docs + fixtures	Divyansh / Bhavya	/docs/API.md, /fixtures	Block 0	Matches contracts
rules.json, conversions.json, facilities.json	Riya / Divyansh	/data	C2	Sourced; unsourced → MANUAL_REVIEW
Source register	Aarvi / Riya	/docs/SOURCES.md	20:00	URL, access date, licence, verified flag
Unit and integration tests, negative controls	Riya / Bhavya	/core, /api	C3	CI green
Evaluation results	Riya / Divyansh	/eval/results.md	Oct 11 10:00	Produced by script; only measured numbers
Known limitations	Divyansh	README	Oct 11 10:00	States: decision support, modelled quantities, facility stock unverified, demo evidence synthetic

Deliverable	Owner / support	Location	Deadline	Acceptance
Measured vs modelled vs synthetic labelling	Divyansh	UI + README	20:00	Every figure tagged

---

7. Time-blocked schedule

DTU runs 8 AM–8 PM. Times below are targets; if you start late, apply the cut line and keep the checkpoint.

Block	Work	Checkpoint — minimum working result	If behind, cut
0 • first 30 min	Audit (repo exists, history from Oct 8, Textract JSON cached?); decide D1–D12; freeze contracts; assign branches	/contracts/v1 + fixtures merged; everyone has their first task	Skip nothing; this block prevents rework
1 • until 12:00	Parallel: Bhavya SAM + API + Step Functions; Riya rules/normalise/classify; Aarvi S1–S4 in mock mode; Divyansh gap + ledger + mentor session 1 (AWS)	**C1 10:30:** deployed API returns fixtures; UI shows S1–S4 from fixtures. **C2 12:00:** real PDF → Textract → lines in review table, or rows path working	Bedrock (never built); facility distance → static list
2 • 12:00–16:00	Evaluate, gap, approve, plan, ledger wired end to end; Riya's labelled set; civil mentor session	**C3 16:00:** whole flow on deployed stack with the demo BOQ, ending in a VERIFIED delivery	Printable PDF → CSV only; extra evidence checks beyond slip uniqueness and S3 existence
3 • 16:00–18:00	Hardening; run /eval; negative-control gate; clean-browser run	Eval run complete; zero unsafe results	UI polish

Block	Work	Checkpoint — minimum working result	If behind, cut
4 • 18:00–20:00	Second mentor critique; AWS proof pack; draft screen recording as a backup	**C4 20:00 feature freeze**	Anything not demonstrable
Oct 11 08:00–10:00	Signed-out smoke test; recheck official deadline	Stack healthy; app loads	—
10:00–12:00	Final eval numbers into README; write-up	README claims audited	Blog
12:00–14:00	Record demo; edit ⩽2:55	Final cut exists	Extra B-roll
14:00–15:00	Upload to YouTube; test signed-out; submit	Submitted by **15:00**	—
15:00–18:00	Emergency buffer only; recheck official time	—	—

Never cut: source clause per recommendation; gap calculator; evidence-gated ledger; human approval; negative-control gate. **If Textract fails on the real PDF:** run the demo through the rows path (still Step Functions, Lambda, DynamoDB) and show a real Textract run on a simpler file, stating that in the narration. Do not present a cached run as live.

---

8. Testing, safety and evaluation criteria

- **Unit tests:** ⩾25 in /core (rules, normaliser, classifier), ⩾8 for gap, ⩾10 for ledger.
- **Contract tests:** every endpoint's response validates against /contracts schemas; every fixture validates.
- **Integration test:** scripted run — create job → rows start → confirm → evaluate → approve → plan → order → delivery with evidence → impact; asserts totals.
- **Safety gate:** negative controls (prestressed, M30+ RCC, retaining wall, water-retaining, unknown grade, unknown unit, missing factor) must produce zero ELIGIBLE or CONDITIONAL. Any failure blocks submission.

- **Failure cases:** merged cells, missing unit, swapped unit/qty, duplicate item numbers, empty BOQ, Textract timeout, S3 evidence missing, delivery above planned + 5%, duplicate slip number, edit after lock.
- **Evaluation metrics:** extraction cell accuracy (item, unit, quantity); classification precision and recall; negative-control result; median upload-to-review time; manual corrections needed on the demo BOQ. Report only measured values.
- **Labels:** real measurement vs modelled quantity vs synthetic demo evidence [S] must be visible in the UI and README. Demo delivery evidence is synthetic unless it is a real transaction.
- **Regulatory assumptions to flag in the README:** gap denominator [A]; schedules from secondary summaries [V, confirm Gazette]; facilities [U]; IS 383 limits from secondary copies [V].

---

9. Mentor checklists

AWS mentors

- Is async Textract with Step Functions polling the right pattern, or should we use SNS completion events? Tips for merged cells and multi-page BOQ tables?
- Review our state machine: retries, catch paths, pagination of GetDocumentAnalysis.
- IAM: is the role split safe? Anything risky in CORS, presigned URLs or the demo passcode approach?
- Deployment: SAM plus Amplify in ap-south-1 — any region limits?
- Cost: Textract table pages, free-tier limits, caching, budget alarm.
- What AWS evidence do judges find convincing beyond screenshots?

Civil / environmental engineering mentor

- Are the IS 383:2016 Table 1 limits correctly read, including Note 2 and the prestressed exclusion?
- Do the percentages apply by mass or volume, and how should aggregate quantity be derived from concrete lines?
- What is the right denominator for the Second and Third Schedule targets, and what is excluded?
- Which standards and clauses cover paver blocks, kerbstones, sub-base and backfill with recycled material?

- Which line types must always stay manual (structural elements, water-retaining, foundations in aggressive soils)?
- Which Delhi plants currently supply which products?
- Is a draft 'waste utilisation plan' format available?

Record answers in /docs/MENTOR_NOTES.md. Any rule the mentor cannot confirm becomes MANUAL_REVIEW.

---

10. Demo storyboard and judge talking points

Recorded by Divyansh (narration and screen); edited by Aarvi; ⩽2:55.

Time	Show	Say
0:00–0:15	Title, one data card	Rules in force since April; Delhi has plants; only 14% of recycled products were used last year
0:15–0:30	A BOQ page	A procurement engineer must read hundreds of lines to find where recycled material is allowed
0:30–1:00	Upload; Step Functions progress; review table; fix one cell	Textract reads the PDF; a person confirms
1:00–1:35	S4: one ELIGIBLE with source clause; one MANUAL_REVIEW structural line	Fixed rules with sources decide; unsure means manual review
1:35–2:00	S5 gap	This BOQ can reach X% of the 5% target (real value from the run) and these lines drive it
2:00–2:20	Approve; export plan; enter PO	A draft plan, not a filing
2:20–2:35	Delivery with evidence; totals	Verified tonnes move only now; evidence is demo data if labelled so
2:35–2:52	Step Functions execution, S3, DynamoDB, CloudWatch with jobId	This is the running system
2:52–3:00	Close	One BOQ line at a time

Talking points: (1) it targets the demand side where the evidence says the bottleneck is; (2) rules are sourced and conservative — AI never decides eligibility; (3) the gap calculator is honest about what standards allow; (4) the ledger separates modelled from verified; (5) AWS runs the pipeline.

Objections and evidence

- 'Planned is not diverted.' → four-state ledger; only VERIFIED claimed; no CO2 claims.
- 'AWS is decoration.' → show a live execution graph and the S3/DynamoDB/CloudWatch trail for one jobId.

- 'Engineers will not trust it.' → source clause per line, human approval, visible assumptions, negative-control results.
- 'Unsafe substitution.' → structural beyond limit is never approvable.
- 'Plant stock is not live.' → capability and distance only; confirm by phone.

---

11. Immediate next actions (in order)

Divyansh: (1) 10-minute state audit and repo-history check; (2) approve D1–D12 and freeze /contracts/v1 + fixtures; (3) start `/core/ledger` and `/core/gap`; (4) review Bhavya's IAM and ASL; (5) book the AWS mentor slot for before noon and a civil mentor slot after.

Bhavya: (1) wait for the freeze, meanwhile write the SAM skeleton (S3, DynamoDB, HTTP API); (2) deploy fixture-returning endpoints; (3) run Textract on the demo PDF and cache the JSON; (4) build the Step Functions machine with the rows path first; (5) set the budget alarm.

Riya: (1) rules.json from the sourced rules only; (2) conversions.json and facilities.json with flags; (3) normaliser and classifier; (4) evaluate and tests; (5) start the labelled set as soon as the demo BOQ is chosen.

Aarvi: (1) scaffold /web with mock mode; (2) S1 and S2 including CSV/XLSX parse and the sample BOQ button; (3) S3 review table; (4) README skeleton, SOURCES.md and THIRD_PARTY_NOTICES.md; (5) S4 and S5 once fixtures are frozen.

Everyone: confirm student verification and Luma registration in chat; post your first DONE/NEXT/BLOCKED/EVIDENCE message at 10:30.